CS 230 Course Notes: Computer Systems

Winter 2026 - Sandy Graham

Talha Yildirim,
tyildir [at] uwaterloo [dot] ca

Contents

1 Assembly Language 3
2 Machine Internals 23

1 Assembly Language

Definition 1.1 (Machine Code)

- Binary code - comprised of 0's and 1's
- "Direct" execution by processor
- Program grouped into instructions
  - fixed v.s. variable length
  - operation code (opcode) + operands
  - Instructions control processor
    - opcode designates operation
    - operands designate data

♣

Definition 1.2 (Assembly Language)

- Human readable "programming language"
  - Very simple compared to Racket, Python, etc
- Almost direct mapping to machine code
  - Except a few concepts
- Assembler turns it into machine code
  - Process is called "assembling" rather than compiling

♣

Definition 1.3 (Instruction Set)

An instruction set is the complete list of machine-level commands a CPU knows how to execute.

- Repertoire of instructions
- Different processors have different sets
  - Many commonalities
    - mathematical
    - memory access
    - control flow

♣

Definition 1.4 (Turing Completeness)

- Fundamental capabilities of instruction set
- Minimum requirements
  - Condition branching (if / goto)
  - Some form of repetition (while / for / recursion)
  - Arbitrary memory manipulation
- Typical programming languages / instruction sets are Turing complete

♣

Definition 1.5 (MIPS Architecture)

- MIPS: Microprocessor without Interlocked Pipeline Stages
- A type of computer architecture
- Multiple revisions, systems, and compilers
  - Not just a single standard MIPS

♣

Definition 1.6 (MIPS Assembly Language)

- Each instruction takes 32 bits
  - 4 bytes = 1 word
- Arithmetic instructions operate on registers
  - 32 registers available number $0 to $31
  - Register $0 always equals 0
- Instructions have up to 3 operands
  - 1st is destination, 2nd and 3rd are sources
  - Same register can be source and destination
- Two addressing modes
  - **Register:** Operands in registers
  - **Immediate:** One operands is a 16-bit constant
- Memory: 32-bit address space
  - Relative and absolute memory load / store
  - Relative and absolute branch instructions

♣

Definition 1.7 (Special Registers)

- **PC**
  - Program counter location (byte address) in machine code
    - incremented by 4 for each instruction
- $0
  - Constant 0
- Register Conventions
  - $29 - stack pointer ($\sp)
  - $30 - frame pointer ($\fp)
  - $31 - return address ($\ra)

♣

Definition 1.8 (Addition)

```
add $3, $2, $11
```

- Add (contents of) register `1` and `2`
- Place result in register `3`
- General notation: `add $d, $s, $t`
- Move (contents of) register `1` to `2`: `add $2, $1, $0`

```
sub $d, $s, $t
```

- Subtract register `t` from `s`
- Place result in register `d`

♣

Definition 1.9 (Debugging)

- **Use output:**
  - Print intermediate values as your program runs so you can see what's happening
- **Early Termination with `jr $31`:**
  - `$31` (also called `$ra`) is the return address register. Executing `jr $31` effectively returns from your program early — it stops execution at that point. The trick is:
  - Insert `jr $31` partway through your code
  - Run the program and inspect the register values at that stopping point
  - Verify each register holds what you expect
  - Move `jr $31` further down and repeat
- General Techniques
  - **Analyze Log Outputs:**
    - Run your program and carefully read any output or error logs the emulator produces. The emulator may show you register states or flag unexpected behavior
  - **Controlled step-by-step execution:**
    - Rather than running the whole program at once, execute it instruction by instruction and check the state after each one. This is what a proper debugger does automatically, but in this environment you simulate it manually using the early termination trick above.

♣

Definition 1.10 (Immediate Addition)

```
addi $t, $s, i
```

- Adds registers `s` and value `i`
- value `i` can be negative - size / range?
- Place results in register `t`
- Use to initialize registers: `addi $t, $0, i`

♣

Definition 1.11 (Multiplication and Division)

Special registers `hi`, `lo`

```
mult $s, $t
```

- Multiply registers `s` and `t`
- Place results in `hi:lo`

```
div $s, $t
```

- Divide register `s` by `t` (integer division)
- Place result in `lo`, remainder in `hi`

```
mfhi $d
```

- Copy contents of `hi` to `$d`

```
mflo $d
```

- Copy contents of `lo` to `$d`

♣

Problem.

Compute average of three numbers

- Values in `$3, $4, $5`
- Result in `$2`

Solution.

```
add $2, $3, $4

add $2, $2, $5

addi $1, $0, 3

div $2, $1

mflo $2
```

Problem.

Compute $(a + b) - (c + d)$

- a, b, c, d in `$1, $2, $3, $4` - preserve values
- results in `$5`

Solution.

```
add $5, $1, $2

add $6, $3, $4

sub $5, $5, $6
```

Definition 1.12 (Conditional Execution)

- Computation dependent on intermediate results
- Otherwise only linear number crunching

♣

Definition 1.13 (Conditional Branching)

`beq $s, $t, i`

- Compare registers `s` and `t`
- `if` equal. skip `i` instructions
- `i` can be negative

`bne $s, $t, i`

- compare registers `s` and `t`
- `if` not equal, skip `i` instructions
- `i` can be negative

The `PC` (Program Counter) The `PC` is a register that holds the address of the next instruction to execute. Normally it just increments by 4 each cycle (since each instruction is 4 bytes).

`if ($s == $t) PC += i *4`

- `PC` increases automatically by 4 when the instruction is fetched and before it is executed
- execution of instruction results in `PC = PC + i * 4`

♣

Definition 1.14 (Set Less Than)

`slt $d, $s, $t`

- compare register `t` and `s`
- `$s < $t` $\rightarrow$ `$d = 1`
- `$s >= $t` $\rightarrow$ `$d = 0`

♣

Example – Loop

Example PC Value	Assembly Instruction
00	`addi $2, $0, 1`
04	`addi $1, $0, 10`
08	`mult $2, $1`
0C	`mflo $2`
10	`addi $1, $1, -1`
14	`bne $1, $0, -4`

Definition 1.15 (Branch Labels)

The Problem Without Labels

```
bne $1, $0, -4
```

That `-4` means "jump back 4 instructions." But what if you add or remove instructions inside the loop? You'd have to manually recalculate that number every time. That's error-prone and annoying.

The Solution: Labels

Instead of writing a raw offset number, you write a named label and let the assembler figure out the offset for you.

```
addi $1, $0, 10      ; $1 = 10

loop:                ; <-- this is just a named marker, not an instruction
  addi $1, $1, -1  ; $1 = $1 - 1
  bne $1, $0, loop ; if $1 != 0, jump back to "loop"
```

The label loop: marks a position in the code. When the assembler sees `bne $1, $0, loop`, it automatically calculates the correct offset number needed to jump back to that position.

♣

Definition 1.16 (Conditional Execution)

This is how you write an `if/else` statement in assembly. There's no `if` keyword — you have to manually redirect the `PC` using branches and labels.

```
; compute something
beq $1, $2, eqcase     ; if $1 == $2, jump to eqcase
                        ; otherwise fall through to the next line

; do something for $1 != $2   <-- the ELSE branch
beq $0, $0, final      ; unconditional jump to final (skip the if-branch)

eqcase:
  ; do something for $1 == $2   <-- the IF branch

final:
  ; continue with program
```

♣

Definition 1.17 (Constants)

Method 1: `addi` (Add Immediate)

```
addi $4, $0, 100
```

This adds `100` to register `$0` (which is always `0`) and stores the result in `$4` — effectively loading the constant `100` into `$4`.

Method 2: `lis` (Load Immediate and Skip)

```
lis $4
.word 0xA3257CE2
```

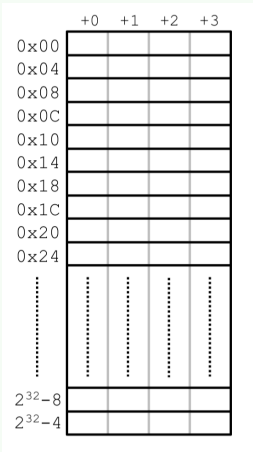
`lis` solves the `16`-bit limitation by using a clever trick:

1. It looks at the very next word in memory (the next `32` bits after the instruction)
2. Loads that full `32`-bit value into a register
3. Then skips over it (so the CPU doesn't try to execute that data as an instruction)

♣

Definition 1.18 (Memory Model)

- **Byte Addressable**
  - Memory is organized as a giant array of bytes, where every single byte has its own unique address
  - With `32`-bit addresses, you can have $2^{32} = 4,294,967,296$ unique addresses
  - Each address points to exactly 1 byte of data
- **Word Aligned**
  - A word in this architecture is 4 bytes (32 bits). While individual bytes have addresses, you can only load or store a full word at a time, and only at specific addresses
- **Unaligned Access Errors**
  - If your program tries to read or write a word from an address that is not a multiple of 4 (e.g., address `0x03`), the CPU cannot do it and throws an "unaligned access" error — the program crashes.



- **The big takeaway:**
  - You address bytes individually, but you read/write 4 bytes (a word) at a time, and that word must start at an address divisible by 4

♣

Definition 1.19 (Memory Access)

`lw $t, i($s)`

- Load word from location `s + i` into `t`
- `s + i` must be word aligned

`sw $t, i($s)`

- store word from `t` into locations `s + i`
- `s + i` must be word aligned

`s + i`

- The memory address to read/write
- `$s` is a register holding a base address (your starting point in memory)
- `i` is a constant offset (how many bytes to shift from that base)
- The CPU adds them together to get the final address

♣

Definition 1.20 (Memory Addresses)

1. **Register Number**

- This is just which register you're referring to — a number between 0 and 31
- `$4, $20, $31` ← these are register numbers

2. **Register Contents**

- This is the actual value stored inside that register — a `32`-bit number
- `$20` might contain the value `0x100`

3. **Memory Address**

- When you write `lw $t, i($s)`, the value `$s + i` is a memory address — it's the location in memory you want to access
- `$s = 0x100, i = 8` memory address = `0x108` ← this is a location, not data

♣

Definition 1.21 (Structures)

A struct is a group of related fields stored sequentially in memory, each `4` bytes apart. If `$s` holds the base address (start of the struct), you access each field using offsets:

```
lw $t, 0($s)   → year        (base + 0)
lw $t, 4($s)   → month       (base + 4)
lw $t, 8($s)   → day         (base + 8)
lw $t, 12($s)  → city        (base + 12)
```

The offsets are always `4` bytes apart because each field is one word (`4` bytes)

Key idea:

`$s` is set once to the start of the record. You never change it — just change the offset `i` to navigate to different fields.

♣

Problem.

You know there is a birth record at address `0x44`, how do i get the month?

Solution.

```
addi $1, $0, 0x44
lw $2, 4($1)
```

or,

```
addi $1, $0, 0x48
lw $2, 0($1)
```

Problem.

What if I want the year and the month?

Solution.

```
addi $1, $0, 0x44
lw $2, 0($1)
lw $3, 4($1)
```

Problem.

How do I set the day to the 15th?

Solution.

```
addi $1, $0, 0x44
addi $2, $0, 15
sw $2, 8($1)
```

Corollary 1.21.1 (Low Level Errors)

- Illegal instructions
- Assignment to read-only register
- Integer division by 0
- Alignment violation
- Memory protection violation
- etc
  - usually result in exception and termination

Definition 1.22 (Assembly File)

- Assembly Instruction
  - The actual commands that tell the CPU what to do
  - You can write numbers in two ways: decimal or hexadecimal
- Label Declarations
  - Labels are named markers pointing to a location in the code. They end with a colon : and let you reference that spot (e.g., for jumps or loops).
- Data Definitions (`.word`)
  - `.word` reserves space in memory and stores a value there (typically a 32-bit integer).
- Comments - start with semicolon

Putting it all together

```
; --- Program Start ---

myNum:  .word 0x20     ; define a word with value 32 (decimal)

start:                 ; label for entry point
    MOV R0, #10        ; load decimal 10 into R0
    MOV R1, #0x1A      ; load hex 26 into R1
    ADD R2, R0, R1     ; R2 = 10 + 26 = 36
```

♣

Definition 1.23 (Input / Output)

In assembly, there are no `print()` or `input()` functions. Instead, special memory addresses are used to communicate with the keyboard and screen.

- Input — Keyboard
  - Address: `0xFFFF0004`
  - To read keyboard input, you load from this address
  - Each load retrieves one character at a time
  - It's a "magic" address — every time you read it, you get a new character
- Output — Screen
  - Address: `0xFFFF000C`
  - To display a character, you write/store a byte to this address
  - Characters are encoded in ASCII
  - Only the least significant 7 bits are used (standard ASCII range: 0–127)

> ⚠ **Warning**
>
> You must use actual keyboard input — input redirection (e.g., piping from a file) may not work correctly

♣

Example.

I/O example

```
lis $1
.word 0xFFFF0004      ; $1 = keyboard input address

lis $2
.word 0xFFFF000C      ; $2 = screen output address

addi $3, $0, 0x1B     ; $3 = 27 (ESC in ASCII)

loop:
    lw $4, 0($1)      ; read one character from keyboard → $4
    sw $4, 0($2)      ; write that character to screen
    bne $3, $4, loop  ; if character ≠ ESC, go back to loop
```

Key Takeaways

- The program is essentially a live echo loop — whatever you type appears on screen
- It reads and writes one character at a time
- The only exit condition is pressing `ESC` (`0x1B`)
- `$0` is hardwired to `0` in MIPS — useful as a baseline for `addi`

Definition 1.24 (MIPS Arrays)

An array is a way to store a fixed-length sequence of numbers in memory — similar to arrays in Python, Java, or C.

- **Element / Item**
  - A single number stored in the array
- **Index**
  - The position of an element (starts at 0)
- **Length / Count / Size**
  - How many elements are in the array

Two Essential Components

1. Address of the 0th Element
   - The memory address where the array starts
   - All other elements are found by offsetting from this address
2. Number of Elements (Length)
   - How many elements the array contains
   - The array is fixed length — you decide the size upfront

♣

Corollary 1.24.1 (Loading the Nth Element)

Load the nth element of the array starting in `$x` into `$y`

- `lw $y, n*4($x)`
- the offset is a constant

Example. Load the 2nd element of the array starting in `$1` into `$3`

- `lw $3, 8($1)`

Definition 1.25 (Stack)

A stack is a dedicated area of memory that works as a Last-In, First-Out (LIFO) structure — like a stack of plates, the last thing you put on is the first thing you take off.

Key Conventions

- Direction
  - Grows downward in memory (address decreases as stack grows)
- Stack Pointer
  - `$29` in standard MIPS
  - `$30` in our simulator
- What it points to
  - The last used word (top of stack)

Corollary 1.25.1 (Push — Saving a Register onto the Stack)

To save a register, you make room first, then write:

```
addi $30, $30, -4     ; 1. decrement stack pointer (make room)
sw $x, 0($30)         ; 2. store register value at new top of stack
```

Corollary 1.25.2 (Pop — Restoring a Register from the Stack)

To restore a register, you read first, then free the space:

```
lw $x, 0($30)         ; 1. load value from top of stack
addi $30, $30, 4      ; 2. increment stack pointer (free space)
```

Corollary 1.25.3 (Multiple Push / Pop)

Saving Three Registers at Once

```
addi $30, $30, -12    ; make room for 3 words (3 × 4 = 12)
sw $3, 0($30)         ; $3 at offset 0 (top)
sw $4, 4($30)         ; $4 at offset 4
sw $5, 8($30)         ; $5 at offset 8 (bottom)
```

Restoring Only One (and Discarding the Others)

```
lw $7, 4($30)         ; grab just the value that was $4
addi $30, $30, 12     ; free all 3 slots at once
```

Caution

If you push 12 bytes, you must pop 12 bytes. Failing to do this corrupts the stack and will cause crashes or undefined behavior.

Corollary 1.25.4 (Stack Frame & Frame Pointer)

Problem.

The stack pointer `$30` can change during a subroutine (e.g. when pushing more values). This makes it unreliable as a reference point inside a function.

Solution.

Frame Pointer

At the start of a subroutine, take a snapshot of the stack pointer and store it as the frame pointer:

```
; at subroutine entry:
add $29, $0, $30      ; frame pointer ($29) = current stack pointer
```

Now `$29` stays constant for the entire routine, giving you a stable base to access local variables:

```
lw $t, 0($29)         ; access variable at offset 0
lw $t, 4($29)         ; access variable at offset 4
lw $t, 8($29)         ; access variable at offset 8
```

Definition 1.26 (Selection Sort - MIPS)

What is selection sort?

Repeatedly find the minimum element in the unsorted portion and swap it to the front.

Warning

COME BACK TO THIS

im to lazy rn

Definition 1.27 (Subroutines)

A subroutine is a reusable block of code that performs a specific task. You may know them as Functions (C, Python, etc)

The key benefits are modularity (break big problems into smaller pieces) and reusability (write the code once, call it many times).

Corollary 1.27.1 (The Core Challenges)

Two problems need to be solved to make subroutines work:

1. **Call / Return**
   - How do you jump to the subroutine, and how does execution know where to come back to after it's done?
2. **Argument / Result Passing**
   - How do you send data into the subroutine, and get results back out?

♡

Definition 1.28 (jal - Calling a Subroutine)

`jal x`

`jal x` stands for Jump And Link. It does two things simultaneously:

1. Saves your return address → copies (current `PC + 4`) into register `$31`
2. Jumps to the function → sets the `PC` to the address of label `x`

`PC + 4` is the address of the next instruction after the `jal` — i.e., where you want to return to after the function finishes. Register `$31` is the conventional "return address" register.

♣

Definition 1.29 (jalr - Indirect Calling)

`jalr $q` stands for Jump And Link Register. It works the same as `jal`, except the destination address comes from a register instead of a label:

1. Saves your return address → copies (current `PC + 4`) into register `$31`
2. Jumps to the function → sets the `PC` to whatever value is stored in register `$q`

To use `jalr`, you first load the function's address into a register using `lis`:

```
lis $1
.word addTwoNumbers   # $1 now holds the address of addTwoNumbers
jalr $1               # jump to that address
```

The value stored in `$q` is called a function pointer — a variable that holds the address of a function. This is powerful because it lets you pass functions as parameters (e.g., passing a comparator function into a sort routine).

♣

Corollary 1.29.1 (Returning from a Subroutine)

Returning is done with `jr $31` — Jump Register to the address saved in `$31`. Since `jal/jalr` stored `PC + 4` there, this sends execution right back to the instruction after the original call.

♡

Example.

```
# --- The function definition ---
addTwoNumbers:
    add $2, $4, $5    # $2 = $4 + $5  (result in $2, inputs in $4 and $5)
    jr $31            # return to caller


# --- Calling it directly with jal ---
jal addTwoNumbers     # $31 = PC+4, then jump to addTwoNumbers
                      # execution resumes here after jr $31


# --- Calling it indirectly with jalr ---
lis $1
.word addTwoNumbers   # load address of function into $1
jalr $1               # $31 = PC+4, then jump to address in $1
                      # execution resumes here after jr $31
```

Example. Print a Null-Terminated String

```
;; Register Convention:
;; $4       - argument: address of string (input, do not modify)
;; $8       - local: output device address (0xffff000c)
;; $9       - local: current string pointer (walks through string)
;; $10      - local: current character being read/printed
;; $31      - return address (set by jal, used by jr $31)

pr_str:
  lis $8
  .word 0xffff000c    ; $8 = address of output device
  addi $9, $4, 0      ; $9 is our moving pointer
                      ; it will walk through the string character by character
loop:
  lw $10, 0($9)       ; load the word at address $9 into $10
  beq $10, $0, end    ; if $10 == 0, jump to end.... i.e NULL
  sw $10, 0($8)       ; store $10 to the output address
  addi $9, $9, 4      ; advance the pointer to the next character
  beq $0, $0, loop    ; 0 == 0 is always true, so always jump to loop

end:
  addi $10, $0, 0xA   ; $10 = 10 (ASCII for newline / line feed)
  sw $10, 0($8)       ; print the newline character
  jr $31              ; return to caller
```

Key Takeaways:

- The function walks through memory `4` bytes at a time, loading one character per word
- It stops when it finds a `0` (`NUL`) — that's the end-of-string signal
- Writing to `0xffff000c` is how you output to the screen in this environment
- `beq $0, $0` , label is the standard `MIPS` idiom for an unconditional branch
- Always ends with `jr $31` to return to the caller

Definition 1.30 (Argument Passing)

When a subroutine (function) is called, the program needs a way to:

1. send inputs (arguments) to the function
2. get the result back from the function

To do this, MIPS uses registers and sometimes the stack.

- There must be an agreement who is caller and callee

Corollary 1.30.1 (MIPS Convention)

- First 4 arguments in registers
- Remainder on the stack

Corollary 1.30.2 (Always Zero — $0)

This register is hardwired to the value 0 and cannot be changed. It's useful any time you need a zero — for comparisons, initializing values, or no-op moves. Writing to it does nothing

Corollary 1.30.3 (Assembler Temporary Reserved — $1)

For the assembler itself. You shouldn't touch this one. The assembler uses it behind the scenes when it needs a scratch register to implement certain instructions

Corollary 1.30.4 (Return Values — $2, $3)

When a function finishes and needs to hand a result back to the caller, it puts that value here. Think of these as the "output slot" of a function

Corollary 1.30.5 (Arguments — $4 to $7)

When you call a function and need to pass inputs to it, you load them into these registers first. So `$4` holds the 1st argument, `$5` the 2nd, and so on (up to 4 arguments this way; more go on the stack)

Corollary 1.30.6 (Temporaries — $8 to $15, $24, $25)

Free-for-all scratch space during computation. The catch: the caller is responsible for saving these if it needs them preserved across a function call, because the callee (the function being called) is free to overwrite them

Corollary 1.30.7 (Saved Temporaries — $16 to $23)

Similar to temporaries, but with the opposite rule: the callee must save and restore these before returning. So if your function uses `$16`, it must push the original value to the stack first and pop it back before it exits. The caller can always trust these will be unchanged after a function call

Corollary 1.30.8 (OS Kernel Reserved — $26 to $27)

Off-limits for regular programs. The operating system uses these exclusively for handling interrupts and exceptions.

Corollary 1.30.9 (Global Pointer — $28)

Points to the middle of a 64KB block of memory where global variables are stored. Having a fixed pointer here lets the CPU access globals efficiently in a single instruction rather than needing to calculate their address each time.

Corollary 1.30.10 (Frame Pointer — $30)

Points to the base of the current function's stack frame. While the stack pointer moves around during a function, the frame pointer stays fixed — giving you a stable reference point to find local variables and saved registers. Some compilers skip this and just use `$sp` directly.

Corollary 1.30.11 (Return Address — $31)

When you call a function with `jal` (jump and link), the CPU automatically saves the return address here — i.e., where to jump back to when the function is done. The callee must save this to the stack if it calls any further functions, otherwise it gets overwritten.

Definition 1.31 (Caller vs Callee Registers)

Callee-saved ($16 – $23) → the function being called saves and restores registers before using them

Caller-saved ($8 – $15, $24, $25) → the function doing the calling saves registers before making a call, in case they get overwritten

> **Remark**
>
> Why callee-saving is more efficient:
>
> There are typically more callers than callee's. If the callee saves once, every caller benefits — less total code than every caller saving individually.

Definition 1.32 (Local Variables)

When a function needs local variables, they are stored on the stack by default. You make room by moving the stack pointer (`$sp`) down when you enter the function, and reclaim that space when you leave.

No restrictions on:

- How many local variables you have
- How large they are (arrays, structs, etc.)

Automatic Variables:

- Function is entered
  - Stack space is reserved (`$sp` moves down)
- Function exits
  - Stack space is released (`$sp` moves back up)

♣

Lemma 1.33 (Preserving the Stack)

A function must preserve the value of the stack pointer

- Always add back any value back any value you subtracted from the stack pointer `$sp` before ending a function
- The stack pointer `$sp` should have the same value at the beginning and end of a function

♡

Example.

Printing Integer

The goal is to take an integer in `$4` and print it character by character to the output device.

Register Map:

- `$4` → Input integer
- `$8` → Memory-mapped output address (0xffff000c)
- `$9` → Working copy of the value
- `$10` → Temporary (current digit, etc.)
- `$11` →Counter (how many digits on stack)
- `$30` → Stack pointer

```
; Part 1 — Setup & Handle Negative

pr_int:
  lis $8
  .word 0xffff000c       # $8 = output device address

  addi $9, $4, 0         # $9 = copy of input value
  addi $11, $0, 0        # $11 = digit counter, start at 0

  slt $10, $9, $0        # is $9 < 0?
  beq $10, $0, comp      # if not negative, skip ahead

  addi $10, $0, 0x2D     # 0x2D = ASCII '-'
```

```
  sw $10, 0($8)          # print the minus sign
  sub $9, $0, $9         # make $9 positive (negate it)



; Part 2 — Extract Digits (Push to Stack)

comp:
  addi $11, $11, 1       # increment digit counter

  addi $10, $0, 10
  div $9, $10            # divide working value by 10

  mfhi $10               # remainder = last digit
  addi $30, $30, -4      # make room on stack
  sw $10, 0($30)         # push digit onto stack

  mflo $9                # quotient = remaining number
  bne $9, $0, comp       # if more digits remain, loop


Part 3 — Pop Digits & Print

output:
  lw $10, 0($30)         # pop digit from stack
  addi $30, $30, 4       # move stack pointer back up

  addi $10, $10, 0x30    # convert digit to ASCII (0→'0', 1→'1', etc.)
  sw $10, 0($8)          # print the character

  addi $11, $11, -1      # decrement counter
  bne $11, $0, output    # if digits remain, loop

  addi $10, $0, 0xA      # 0xA = ASCII newline (LF)
  sw $10, 0($8)          # print newline

  jr $31                 # return
```

Definition 1.34 (Recursion)

Mathematical Programming technique

- Divide larger problem into smaller ones
- Recursion mandates stack
  - need memory area per subroutine

Equivalent to
- loop
- conditional branch

♣

Example.

Factorial in MIPS

Two approaches:

1. Recursive
2. Iterative

Register Map:

- `$2` → Running result (starts at 1)
- `$3` → Temporary comparison
- `$4` → Current n (counts down)
- `$31` → Return address (set by jal)

Recursive Approach:

```
addi $2, $0, 1        # result = 1 (identity for multiplication)

fac:
    slt $3, $0, $4    # is 0 < $4? (is n > 0?)
    beq $3, $0, end   # if not, we're done — return

    addi $30, $30, -8 # make room for 2 values on stack
    sw $31, 4($30)    # save return address
    sw $4, 0($30)     # save current n

    addi $4, $4, -1   # n = n - 1
    jal fac           # call fac(n-1), $31 gets overwritten here

r:  lw $31, 4($30)    # restore return address
    lw $4, 0($30)     # restore n
    addi $30, $30, 8  # clean up stack

    mult $2, $4       # result = result × n
    mflo $2           # store result

end:
    jr $31            # return
```

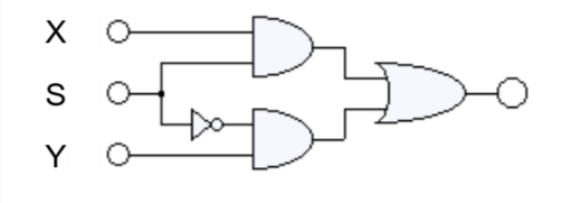
Iterative Approach:

```
addi $2, $0, 1        # result = 1
slt $3, $0, $4        # is n > 0?
beq $3, $0, end       # if n=0, skip loop

addi $3, $4, 0        # $3 = copy of n (use $3 as counter)

loop:
    mult $2, $3       # result = result × $3
    mflo $2           # store result
    addi $3, $3, -1   # decrement counter
    bne $3, $0, loop  # if counter ≠ 0, loop again

end:
    jr $31            # return
```

2 Machine Internals

Definition 2.1 (Multipexer)

- Simplest control element

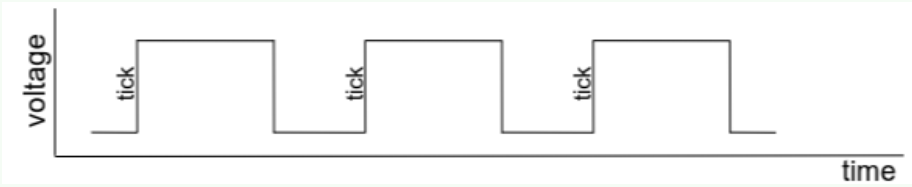


- Forward `X` or `Y` signal, depending on `S`

♣

Definition 2.2 (Clock)

- Clock cycle: Beat of computer
- Clock signal



- Electrical signals propagate fast
  - but not infinitely fast: gate delays
- Rising edge called a tick, keeps things in sync

♣

Definition 2.3 (Cycle Execution)

- One instruction per clock cycle
  - Fixed cycle length must cover slowest instruction

1. One Instruction Per Clock Cycle
   - Every instruction must finish in exactly one clock cycle
   - The problem? Different instructions take different amounts of time physically. A memory load (lw) is much slower than a simple add
2. One Instruction, Multiple Clock Cycles
   - Instead of one fixed cycle length, each instruction is broken into subtasks (stages), and each subtask takes one cycle
   - Fast instructions finish in fewer cycles
   - Slow instructions take as many cycles as they need
   - No wasted time padding fast instructions

♣